VISVESVARAYA TECHNOLOGICAL UNIVERSITY

Jnana Sangama, Machhe, Belagavi – 590018, Karnataka

2022 SCHEME – BCS714D

ASSIGNMENT 1 REPORT

ON

Distributed Food and Nutrient Analytics Using MongoDB, Hadoop MapReduce, Pig and Hive

Track B – Part 1: Three Core Themes

Submitted as part of the coursework for Big Data Analytics

Sl. No.	Student name	USN	Marks	Signature
1	Abhishek R	1JS23CS004		
2	Adithya V	1JS23CS005		
3	Dhanush Singh G	1JS23CS048		

Group No.: 18 | Section: CSE A

Course Faculty: Sandhiya B

Department of Computer Science and Engineering

JSS ACADEMY OF TECHNICAL EDUCATION

JSS Campus Road, Dr. Vishnuvardhan Road, Bengaluru – 560060

Academic Year: 2026–2027 (ODD Semester)

Contents

Section	Page
1. Introduction, Objectives and Dataset	3
2. Environment and Architecture	4
3. Cluster Verification	5
4. MongoDB Ingestion and Linkage	6
5. MongoDB Aggregation and Profiling	7
6. HDFS File Management	8
7. MapReduce Analytics on YARN	9
8. Pig Transformation and Hive Tables	10
9. Hive Performance and Storage Results	11
10. Conclusion and References	12

Dataset: USDA FoodData Central, October 31, 2024 CSV release. Group 18, CSE A. The report presents the implementation and recorded measurements for the linked 50,000-food development dataset.

1. Introduction, Objectives and Dataset

This project analyses the USDA FoodData Central October 2024 CSV release using MongoDB, Hadoop MapReduce, Pig and Hive. Food descriptions, nutrient definitions and amounts are linked through identifiers and processed into food-type counts and nutrient summaries. The Flask dashboard presents the recorded evidence as the Track B practical extension.

The objectives are to ingest and index documents, execute aggregation pipelines, demonstrate distributed file operations and MapReduce, transform prepared data using Pig, and compare identical Hive queries over Text and RCFile storage.

Dataset item	Full release	Analysed subset
Foods	2,063,876	50,000
Food–nutrient observations	26,848,323	1,126,928
Nutrient definitions	477	477
Source tables	34 CSV files	3 principal linked tables

1.1 Selection and source relationships

The development subset takes the first 50,000 foods by source-row ordinal and retains all their linked nutrient observations. It is a deterministic development selection, not a representative statistical sample. Full-source ingestion separately imported all 34 CSV tables into MongoDB.

food.csv supplies food IDs, names and types; food_nutrient.csv connects food IDs to nutrient IDs and amounts; nutrient.csv supplies nutrient names and units. The joins use fdc_id and nutrient_id. Each prepared record contains food ID, type, category, nutrient ID, name, unit and amount.

1.2 Interpretation

A food can have many nutrient observations. A grouped mean is the total recorded amount divided by the number of usable observations for that nutrient and food type. It does not describe a single named product or an amount consumed.

2. Environment and Architecture

The reported experiment uses one master and two Hadoop worker containers connected through Docker on an Ubuntu 24.04 / WSL2 laptop. Approximately 10 GB RAM and four processors are available to WSL. This is a three-container development cluster, distinct from three physical lab computers.

Component	Configuration
Hadoop / Java	Hadoop 2.10.2; Java 8
Analytics tools	MongoDB 7.0.16; Pig 0.17.0; Hive 2.3.9
Master	NameNode, ResourceManager and Pig/Hive clients
Workers	Two DataNodes and two NodeManagers
Storage	Persistent directories; HDFS replication factor 2

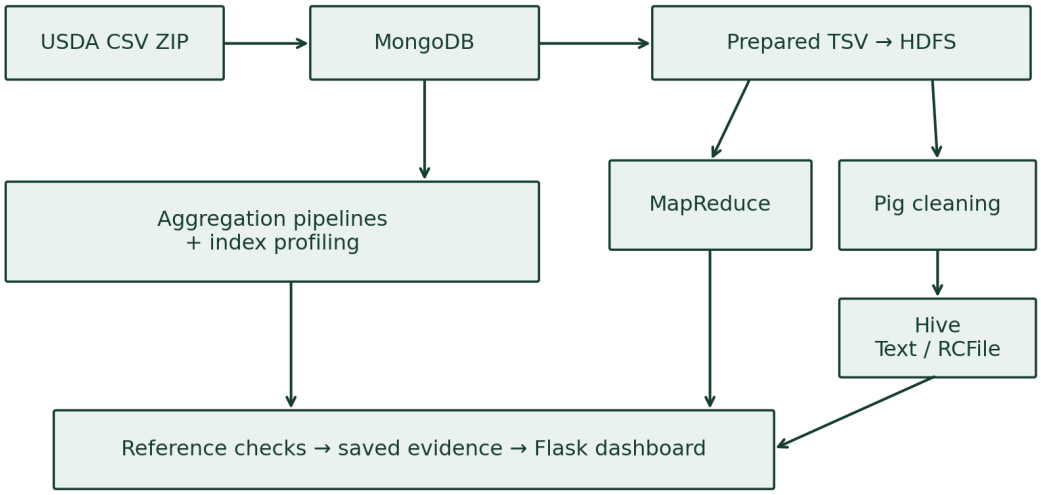


Figure 1. Dataset processing branches and evidence flow.

MongoDB analytics and Hadoop analytics use the same selected records. MapReduce and Pig read prepared HDFS files as separate branches; Pig writes the cleaned input used by Hive. Service switching reduces concurrent memory use on the laptop.

3. Cluster Verification

The supplied terminal evidence reports two live DataNodes, worker1 and worker2, with normal decommission status. These are Docker network addresses. The capacity shown by each container reflects its underlying filesystem and should not be added as independent physical disk capacity.

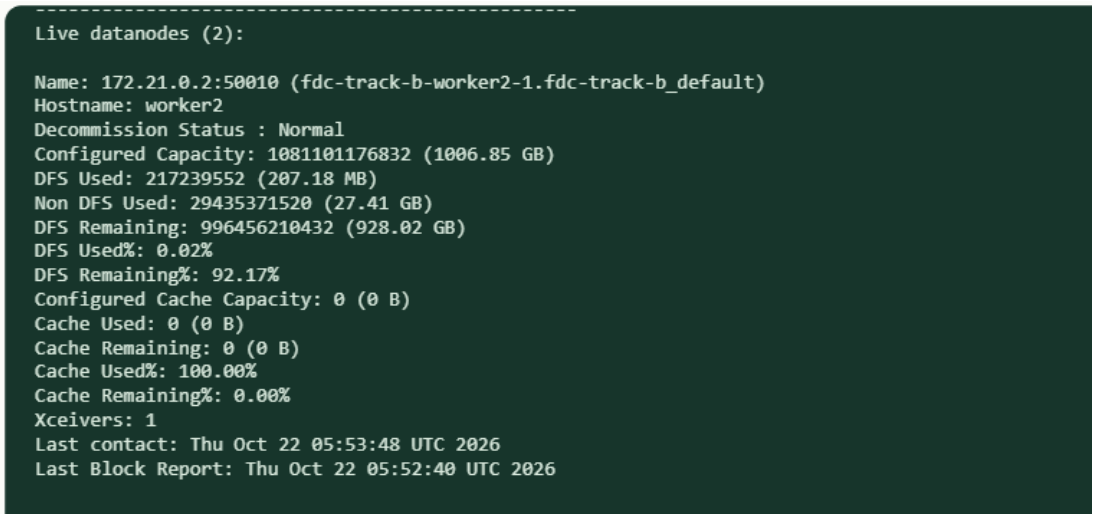


Figure 2. HDFS report identifying two live DataNodes and worker2.

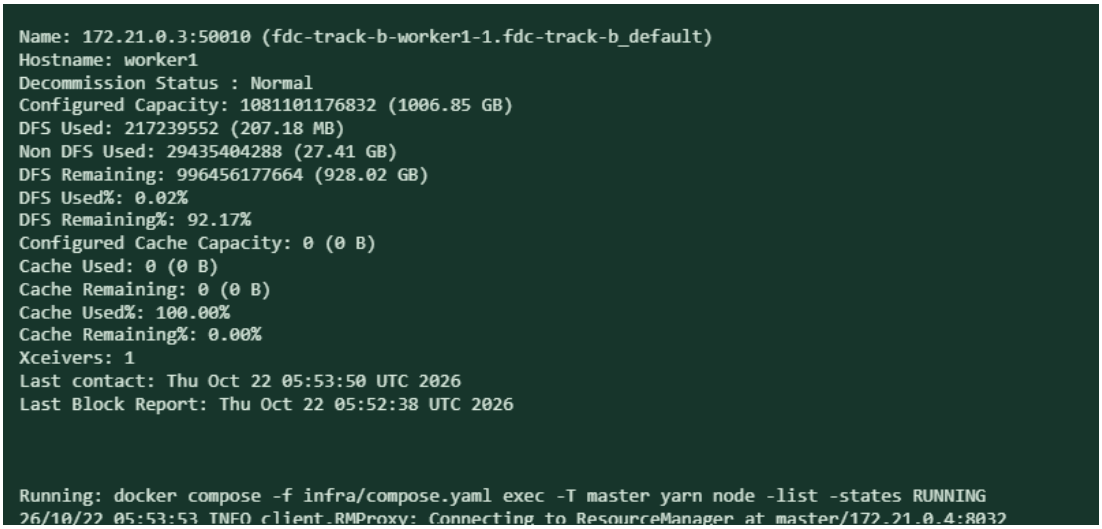
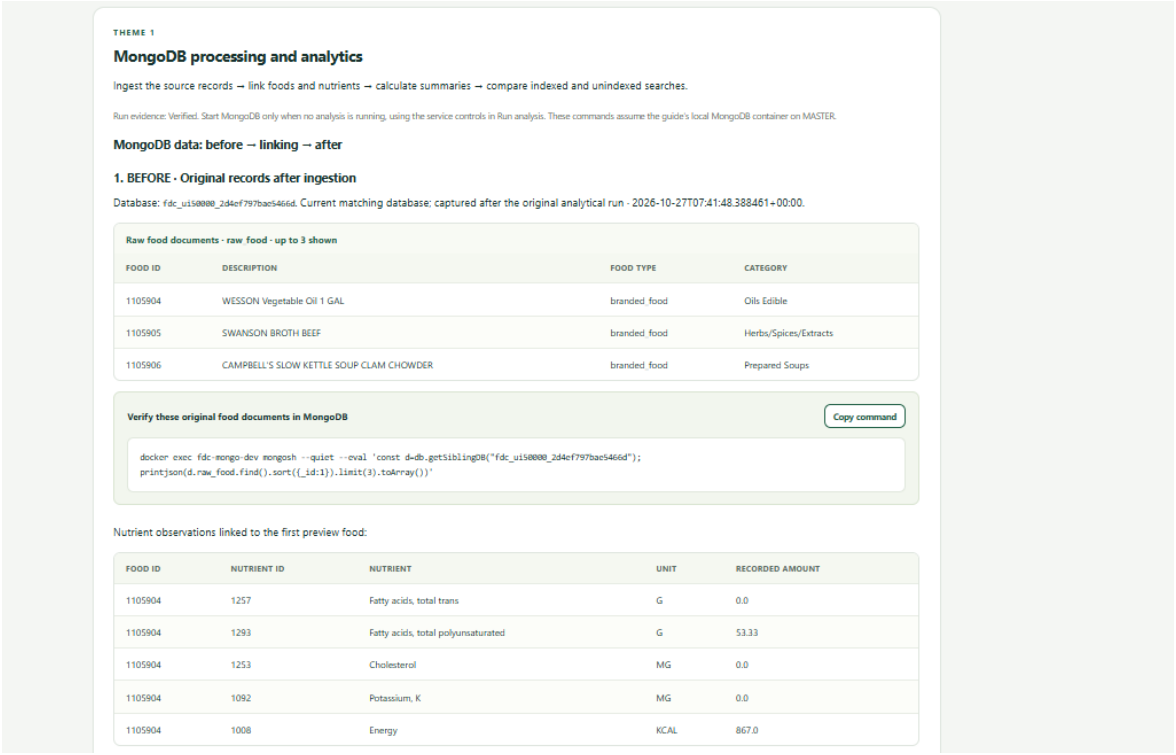


Figure 3. Worker1 details and the YARN verification command.

These captures document a cluster verification session. The selected analytical run separately records two DataNodes and two YARN workers in its dashboard evidence. A sample Pi job is used to verify MapReduce execution before the food analytics job; Pi is not a nutritional result.

4. MongoDB Ingestion and Linkage

The subset uses raw_food, raw_food_nutrient and raw_nutrient as source collections. The derived foods collection links each food to an array of nutrient observations. The metadata collection records source identity, row counts, subset selection and import completion.



THEME 1

MongoDB processing and analytics

Ingest the source records → link foods and nutrients → calculate summaries → compare indexed and unindexed searches.

Run evidence: Verified. Start MongoDB only when no analysis is running, using the service controls in Run analysis. These commands assume the guide's local MongoDB container on MASTER.

MongoDB data: before — linking — after

1. BEFORE · Original records after ingestion

Database: `fdc_ui50000_2d4ef797bae5466d`. Current matching database: captured after the original analytical run · 2026-10-27T07:41:48.388461+00:00.

Raw food documents · raw_food · up to 3 shown

FOOD ID	DESCRIPTION	FOOD TYPE	CATEGORY
1105904	WESSON Vegetable Oil 1 GAL	branded_food	Oils Edible
1105905	SWANSON BROTH BEEF	branded_food	Herbs/Spices/Extracts
1105906	CAMPBELL'S SLOW KETTLE SOUP CLAM CHOWDER	branded_food	Prepared Soups

Verify these original food documents in MongoDB Copy command

```
docker exec fdc-mongo-dev mongosh --quiet --eval 'const d=db.getSiblingDB("fdc_ui50000_2d4ef797bae5466d");
printjson(d.raw_food.find().sort({_id:1}).limit(3).toArray())'
```

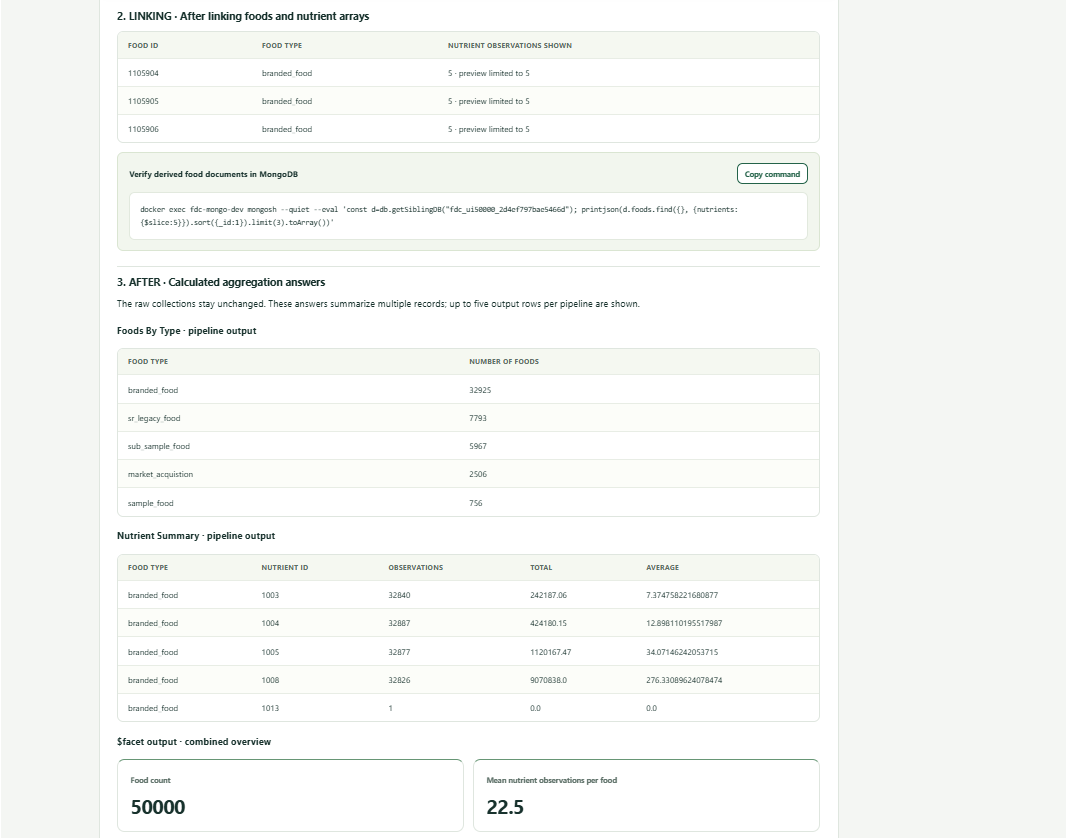
Nutrient observations linked to the first preview food:

FOOD ID	NUTRIENT ID	NUTRIENT	UNIT	RECORDED AMOUNT
1105904	1257	Fatty acids, total trans	G	0.0
1105904	1293	Fatty acids, total polyunsaturated	G	53.33
1105904	1253	Cholesterol	MG	0.0
1105904	1092	Potassium, K	MG	0.0
1105904	1008	Energy	KCAL	867.0

Figure 4. Original food documents and observations linked to the first food.

Food ID 1105904 is WESSON Vegetable Oil 1 GAL. Its preview includes polyunsaturated fatty acids (53.33 G), cholesterol (0 MG) and energy (867 KCAL). These are source observations, not grouped totals. The preview is explicitly labelled as captured later from a matching database; it does not alter the earlier benchmark.

5. MongoDB Aggregation and Profiling



2. LINKING · After linking foods and nutrient arrays

FOOD ID	FOOD TYPE	NUTRIENT OBSERVATIONS SHOWN
1105904	branded_food	5 · preview limited to 5
1105905	branded_food	5 · preview limited to 5
1105906	branded_food	5 · preview limited to 5

Verify derived food documents in MongoDB Copy command

```
docker exec fdc-mongo-dev mongosh --quiet --eval 'const d=db.getSiblingDB("fdc_ui50000_2d4ef797bae5466d"); printjson(d.foods.find({}, {nutrients: {$slice:5}}).sort({_id:1}).limit(3).toArray())'
```

3. AFTER · Calculated aggregation answers

The raw collections stay unchanged. These answers summarize multiple records; up to five output rows per pipeline are shown.

Foods By Type · pipeline output

FOOD TYPE	NUMBER OF FOODS
branded_food	32925
sr_legacy_food	7793
sub_sample_food	5967
market_acquisition	2506
sample_food	756

Nutrient Summary · pipeline output

FOOD TYPE	NUTRIENT ID	OBSERVATIONS	TOTAL	AVERAGE
branded_food	1003	32840	242187.06	7.374758221680877
branded_food	1004	32887	424180.15	12.898110195517987
branded_food	1005	32877	1120167.47	34.07146242053715
branded_food	1008	32826	9070838.0	276.33089624078474
branded_food	1013	1	0.0	0.0

$facet output · combined overview

Food count: 50000

Mean nutrient observations per food: 22.5

Figure 5. Linked food documents, aggregation results and faceted overview.



4. Index performance

The same food lookup is repeated five times per plan. Documents examined means records checked; Returned means matching records found. A 0 ms value is below the timer's millisecond resolution.

PLAN	RUNS	DOCUMENTS EXAMINED	INDEX KEYS	RETURNED	MEDIAN SERVER MS
collection_scan	5	50000	0	1	42
indexed	5	1	1	1	0

Verify the indexed lookup with executionStats Copy command

```
docker exec fdc-mongo-dev mongosh --quiet --eval 'const d=db.getSiblingDB("fdc_ui50000_2d4ef797bae5466d"); const f=d.raw_food.findOne(); if(f) printjson(d.runCommand({explain:{find:"raw_food",filter:{fdc_id:f.fdc_id},hint:"idx_fdc_id"},verbosity:"executionStats"}))'
```

Figure 6. Five profiling repetitions per plan: collection scan versus index.

The pipelines use $match, $group, $project, $unwind and $facet. The protein example has 32,840 branded-food observations, total 242,187.06 G and mean 7.37476 G. The lookup examines 50,000 documents without the index and one with it. Median server times are 42 ms and 0 ms; zero is below timer resolution, not zero work.

6. HDFS File Management

The 50,000-food run stores prepared input and analytical outputs under /user/root/fdc/realdev_1792244956898668695. The HDFS demonstration creates directories, uploads files, reads a probe, downloads and byte-checks it, then deletes only the temporary demo directory.

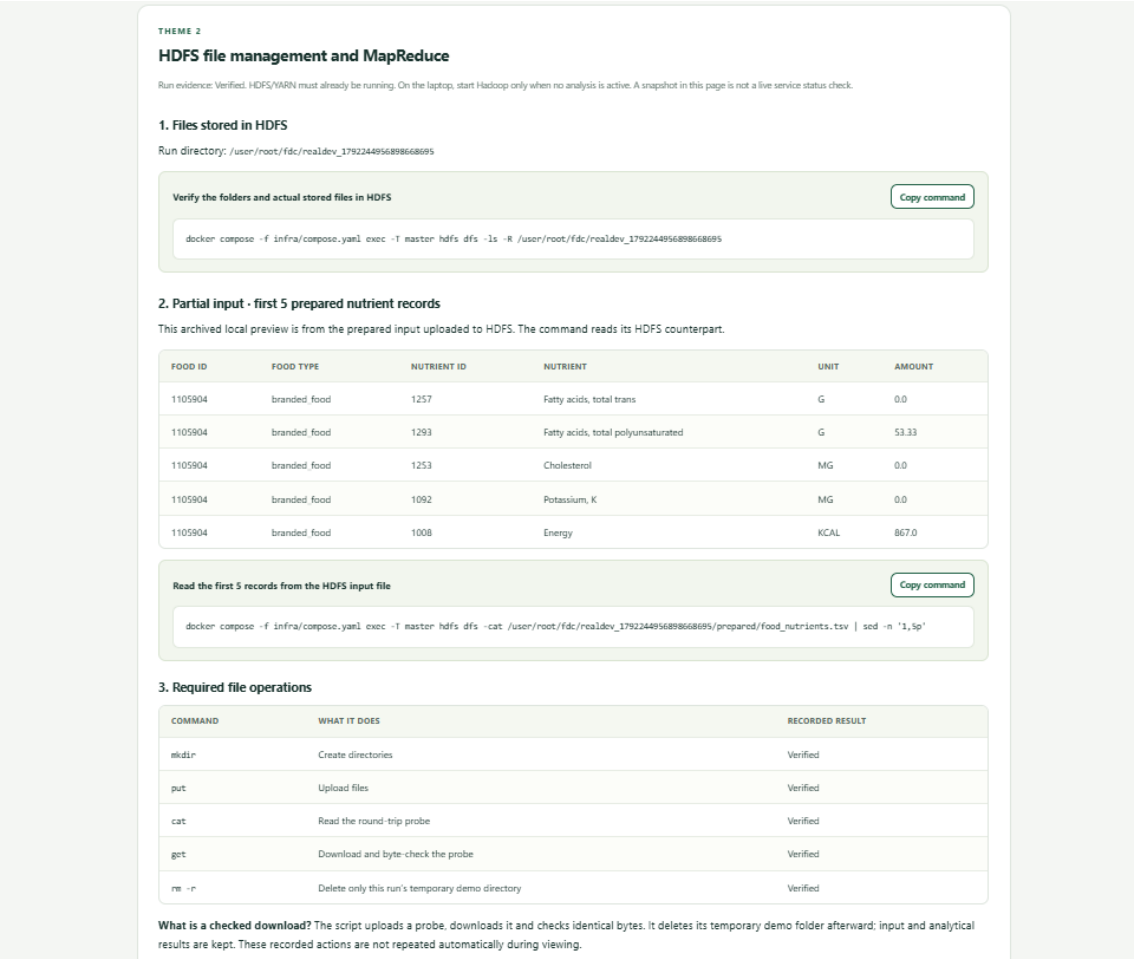


Figure 7. HDFS input samples and verified mkdir, put, cat, get and rm -r operations.

Prepared food and nutrient TSV files are retained as inputs. The mapreduce, pig_clean and hive_rc directories contain reducer output, Pig-cleaned rows and the converted Hive dataset. The adjacent commands inspect those stored files; merely viewing the dashboard does not repeat file operations.

7. MapReduce Analytics on YARN

The Python Streaming mapper reads seven-field records, validates numeric amounts and excludes negative values. It emits a food-type/nutrient key with amount and count. Hadoop partitions and sorts the keys, and each reducer calculates observation count, total and mean without buffering the whole group.

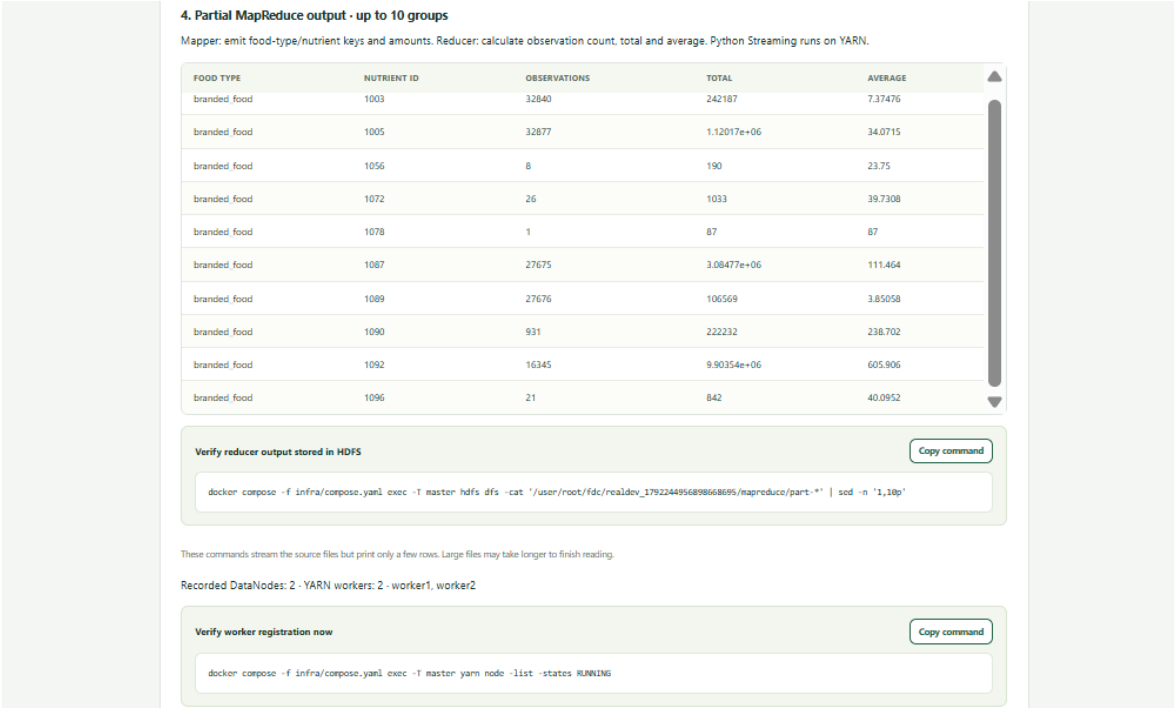


Figure 8. Partial reducer output and recorded worker registration.

For branded_food / protein, MapReduce returns 32,840 observations, total approximately 242,187.06 and mean 7.37476. The screen rounds some numbers for readability; saved numeric outputs are compared using a floating-point tolerance.

Independent reference calculations and MongoDB aggregates are reconciled with the distributed output. This checks that the same records were used and that counts, sums and means agree. Two DataNodes and two YARN workers, worker1 and worker2, are recorded for the selected run.

8. Pig Transformation and Hive Tables

Pig reads the prepared HDFS input independently of MapReduce. It keeps non-null food IDs, nutrient IDs, food types and nonnegative amounts. All 1,126,928 prepared observations remain after cleaning; zero negative values are excluded in this subset.

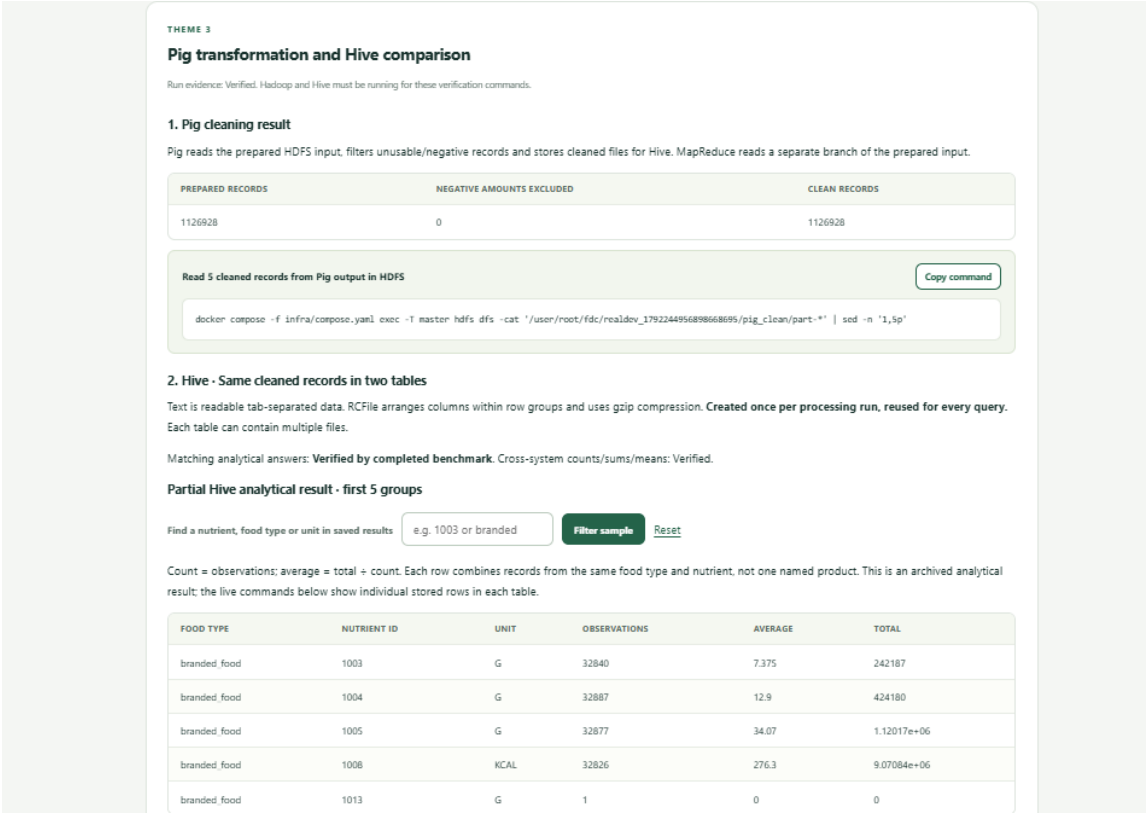
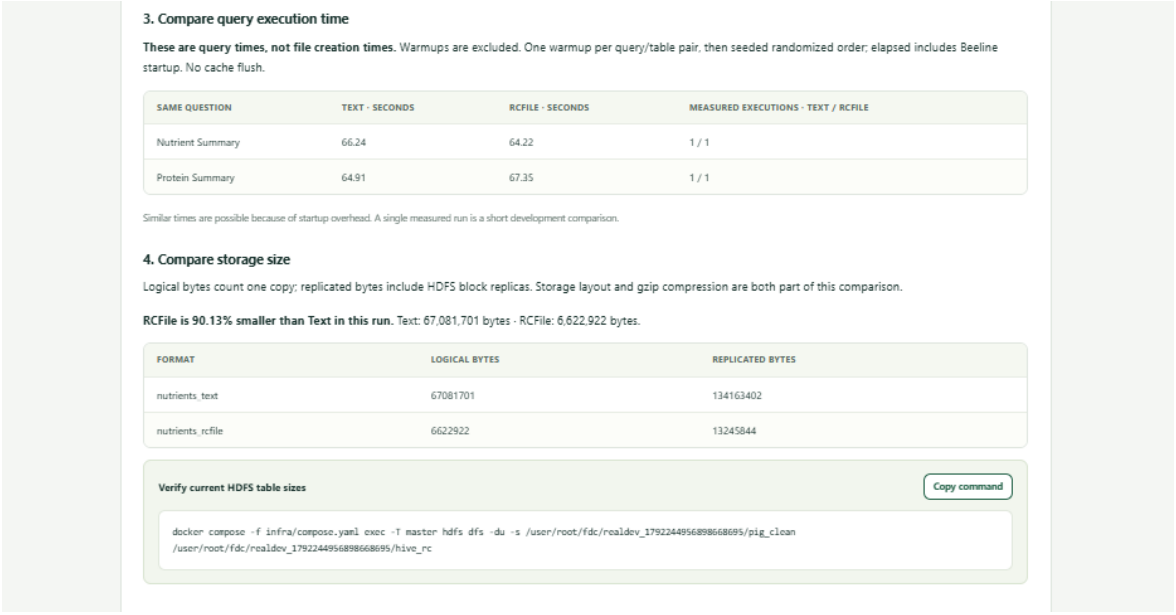


Figure 9. Pig accounting, equivalent Hive datasets and partial analytical results.

nutrients_text points to Pig's tab-separated output; nutrients_rcfile stores the same rows in gzip-compressed RCFile format. These datasets are created once per processing run and reused. Query answers are saved separately as TSV files, with logs and benchmark metadata.

Two questions calculate COUNT, SUM and AVG by food type, nutrient ID and unit: an all-nutrient summary and a protein-only summary (ID 1003, unit G). Both run on both tables, once for warmup and once for measurement: eight executions in total.

9. Hive Performance and Storage Results



3. Compare query execution time

These are query times, not file creation times. Warmups are excluded. One warmup per query/table pair, then seeded randomized order; elapsed includes Beeline startup. No cache flush.

SAME QUESTION	TEXT · SECONDS	RCFILE · SECONDS	MEASURED EXECUTIONS · TEXT / RCFILE
Nutrient Summary	66.24	64.22	1 / 1
Protein Summary	64.91	67.35	1 / 1

Similar times are possible because of startup overhead. A single measured run is a short development comparison.

4. Compare storage size

Logical bytes count one copy; replicated bytes include HDFS block replicas. Storage layout and gzip compression are both part of this comparison.

RCFile is 90.13% smaller than Text in this run. Text: 67,081,701 bytes · RCFile: 6,622,922 bytes.

FORMAT	LOGICAL BYTES	REPLICATED BYTES
nutrients_text	67081701	134163402
nutrients_rcfile	6622922	13245844

Verify current HDFS table sizes Copy command

```
docker compose -f infra/compose.yaml exec -T master hdfs dfs -du -s /user/root/fdc/realdev_1792244956898668695/pig_clean
/user/root/fdc/realdev_1792244956898668695/hive_rc
```

Figure 10. Recorded query times and Text/RCFile storage measurements.

The all-nutrient query takes 66.24 s on Text and 64.22 s on RCFile. The protein query takes 64.91 s and 67.35 s respectively. There is one measured execution per question/table pair; warmups are excluded. Elapsed time includes Beeline startup and no cache flush is performed.

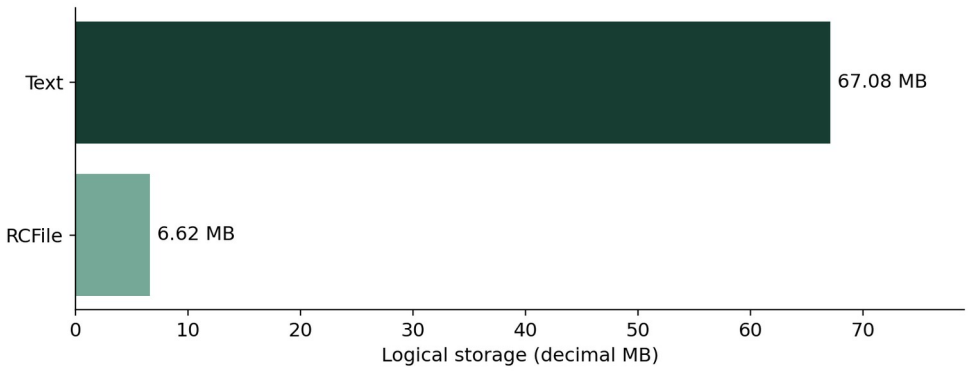


Figure 11. Logical storage for equivalent cleaned input (decimal MB).

Text occupies 67,081,701 logical bytes; RCFile occupies 6,622,922 bytes, approximately 90.13% less. Replicated sizes are twice these values. Both formats return matching answers. RCFile is slightly faster for one query and slower for the other, so this run supports storage savings rather than a consistent speed advantage.

10. Conclusion and References

The implementation covers MongoDB ingestion, indexing and aggregation; HDFS file operations and YARN Streaming; Pig transformation; and Hive Text/RCFile comparison. The linked development dataset contains 50,000 foods and 1,126,928 nutrient observations. Saved evidence connects the inputs to the calculated answers and cross-system checks.

The index reduces documents examined from 50,000 to one. Hive returns matching results across formats, while gzip RCFile reduces logical storage by approximately 90.13%. Measured query times remain similar and vary by question. The workflow is presented through the Flask dashboard described in Assignment 2.

References

1. USDA FoodData Central — Download Datasets. https://fdc.nal.usda.gov/download-datasets/

2. MongoDB — Aggregation Pipelines. https://www.mongodb.com/docs/manual/core/aggregation-pipeline/

3. Apache Hadoop — Cluster Setup. https://apache.github.io/hadoop/hadoop-project-dist/hadoop-common/ClusterSetup.html

4. Apache Pig 0.17 — Documentation. https://pig.apache.org/docs/r0.17.0/

5. Apache Hive — Language Manual. https://cwiki.apache.org/confluence/display/Hive/LanguageManual

6. Project source code. https://github.com/Dhanush-singh-g/BDA-Assignment